

Final Project: Collaborative Python System

Project Overview (5 Points)

Groups will be assigned a real-world task (e.g., Library Management, Inventory Tracker, Financial Record System). The project must follow a modular architecture where group members are responsible for specific classes or packages.

Deliverable: A GitHub repository containing the application code, a requirements.txt file, and a README detailing the architectural design and individual contributions.

1. Foundation & Logic (10 Points)

- **Control Flow:** Implement robust input/output, conditional statements, and logical operators for core menu systems.
- **Loops:** Use while/for loops to process incoming data streams or user menus.
- **Grading:** Correct logic (5 pts), user-friendly interface/CLI (5 pts).

2. Data Management & Collections (8 Points)

- **Collections:** Utilize lists, tuples, sets, and dictionaries to manage system entities (e.g., product sets, user dictionaries).
- **Data Persistence:** Use the OS, CSV, and JSON modules to save and load system state.
- **Grading:** Proper collection usage (4 pts), reliable File I/O (4 pts).

3. Core & Advanced OOP (27 Points)

- **Core OOP (15 pts):** Define classes and objects for primary domain entities; utilize encapsulation for system attributes.
- **Advanced OOP (12 pts):** Implement **Inheritance** (e.g., BaseItem → SpecialItem), **Association** (e.g., Manager manages Entities), and **Polymorphism** to enable flexible system behavior.
- **Grading:** Compliance with OOP principles (15 pts), correct application of class relationships (12 pts).

4. Functions & Functional Programming (8 Points)

- **Modular Logic:** Define clear functions with positional and keyword arguments.
- **Functional Tools:** Apply lambda, map, and filter for efficient data transformation.
- **Grading:** Clean modular code (4 pts), effective use of functional primitives (4 pts).

5. Modules, Packages & Organization (10 Points)

- **Packaging:** Organize modules into directories with __init__.py files.
- **Imports:** Demonstrate mastery of import and from module import function to connect group-authored code.
- **Built-ins:** Utilize modules like math, sys, or random where appropriate.

- **Grading:** Proper package structure (5 pts), logical organization (5 pts).

6. Testing, Decorators & Iterators (16 Points)

- **Unit Tests (8 pts):** Write at least 5 tests using unittest (asserting values, true/false conditions).
- **Decorators (3 pts):** Apply at least one custom decorator (e.g., `@log_action`).
- **Iterators/Generators (3 pts):** Implement a custom iterator or generator for large data processing.
- **Regex (2 pts):** Use the `re` module for input validation.

7. Planning & Real-World Application (16 Points)

- **Project Planning (4 pts):** Submit a design document in the README outlining the class hierarchy and logic flow.
- **Collaboration (4 pts):** Document individual roles and evidence of integration.
- **Quality Assurance (4 pts):** PEP8 compliance, docstrings, and error handling.
- **Execution (4 pts):** 100% pass rate on unit tests and bug-free runtime.

Total: 100 points

Group Assignment Protocol

To manage the assignment effectively:

1. **Assign Domains:** Groups receive a unique problem (e.g., "Hospital Management System" vs. "Retail Stock Analyzer").
2. **Individual Accountability:** Each member acts as the "Lead" for at least one module (e.g., Member 1: DatabaseModule, Member 2: ProcessorModule, Member 3: UserInterfaceModule).
3. **Integration Phase:** Require a `main.py` that imports and orchestrates the distinct modules into a unified system.